ICG 作業調整

再加入一個物件，將袋鼠，與茶壺並列。圖形各佔1/2呈現，而且不能影響到現有功能， 現有的控制 不受影響1.三種著色模式實現 2.光照模型 3.3D變換控制 **4.剪裁平面(Clipping Plane)**: 5.**動畫效果**
我的修改方案如下

- **三種著色模式 -** 不會受影響，兩個模型都可以使用相同的著色器
- **光照模型 -** 不會受影響，同樣的光照計算可以應用於兩個模型
- **3D變換控制 -** 需要調整，但可以維持原有功能
- **剪裁平面 -** 不會受影響，可以同樣應用於兩個模型
- **動畫效果 -** 不會受影響，可以同樣應用於兩個模型

1.**加載袋鼠模型**：
`var kangarooVertexPositionBuffer;`
`var kangarooVertexNormalBuffer;`
`var kangarooVertexFrontColorBuffer;`

```
function handleLoadedKangaroo(kangarooData) {
kangarooVertexPositionBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexPositionBuffer);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(kangarooData.vertexPositions),
gl.STATIC_DRAW);
kangarooVertexPositionBuffer.itemSize = 3;
kangarooVertexPositionBuffer.numItems = kangarooData.vertexPositions.length / 3;
```

```
`kangarooVertexNormalBuffer = gl.createBuffer();`
`gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexNormalBuffer);`
`gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(kangarooData.vertexNormals),
gl.STATIC_DRAW);`
`kangarooVertexNormalBuffer.itemSize = 3;`
`kangarooVertexNormalBuffer.numItems = kangarooData.vertexNormals.length / 3;`

`kangarooVertexFrontColorBuffer = gl.createBuffer();`
`gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexFrontColorBuffer);`
`gl.bufferData(gl.ARRAY_BUFFER, new
Float32Array(kangarooData.vertexFrontcolors), gl.STATIC_DRAW);`
`kangarooVertexFrontColorBuffer.itemSize = 3;`
```

```
  `kangarooVertexFrontColorBuffer.numItems =
  kangarooData.vertexFrontcolors.length / 3;`

}

function loadKangaroo() {
var request = new XMLHttpRequest();
request.open("GET", "./model/Kangaroo.json");
request.onreadystatechange = function () {
if (request.readyState == 4) {
handleLoadedKangaroo(JSON.parse(request.responseText));
}
}
request.send();
}
```

2.修改 **webGLStart()** 函數加載袋鼠模型：

```
function webGLStart() {
var canvas = document.getElementById("ICG-canvas");
initGL(canvas);
initShaders();
loadTeapot();
loadKangaroo(); // 加載袋鼠模型

  `gl.clearColor(ka, ka, ka, 1.0);`
  `gl.enable(gl.DEPTH_TEST);`

  `tick();`

}
```

3. 修改 **drawScene()** 函數以繪製兩個模型：

```
function drawScene() {
gl.viewport(0, 0, gl.viewportWidth, gl.viewportHeight);
gl.clearColor(ka, ka, ka, 1.0);
gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);

  if (teapotVertexPositionBuffer == null ||
      teapotVertexNormalBuffer == null ||
      teapotVertexFrontColorBuffer == null ||
      kangarooVertexPositionBuffer == null ||
```

```
        kangarooVertexNormalBuffer == null ||
        kangarooVertexFrontColorBuffer == null) {
        return;
    }

    // 確保 shadingType 和 clipPlaneX 設置正確
    gl.uniform1i(gl.getUniformLocation(shaderProgram, "vsShadingType"),
    shadingType);
    gl.uniform1i(gl.getUniformLocation(shaderProgram, "fsShadingType"),
    shadingType);
    gl.uniform1f(gl.getUniformLocation(shaderProgram, "clipPlaneX"),
    clipPlaneX);

    // Setup Projection Matrix
    mat4.perspective(45, gl.viewportWidth / gl.viewportHeight, 0.1, 100.0,
    pMatrix);

    // 繪製茶壺 - 放在畫面左側
    drawTeapot(-15, 0, -50); // x座標為負值，將茶壺放在左側

    // 繪製袋鼠 - 放在畫面右側
    drawKangaroo(15, 0, -50); // x座標為正值，將袋鼠放在右側
```

}

// 繪製茶壺的函數
function drawTeapot(posX, posY, posZ) {
// 保存當前矩陣狀態
var mvMatrixCopy = mat4.create();
mat4.set(mvMatrix, mvMatrixCopy);

```
    // Setup Model-View Matrix
    mat4.identity(mvMatrix);
    mat4.translate(mvMatrix, [posX, posY, posZ]);

    // Apply translation transformation
    var transVec = update_trans();
    mat4.translate(mvMatrix, transVec);

    var rotateVec_init = [0,0,0];
```

```
mat4.rotate(mvMatrix, degToRad(rotateVec_init[0]), [1, 0, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec_init[1]), [0, 1, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec_init[2]), [0, 0, 1]);

// Apply rotation transformation
var rotateVec = update_rotate();
mat4.rotate(mvMatrix, degToRad(rotateVec[0]), [1, 0, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec[1] + teapotAngle), [0, 1, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec[2]), [0, 0, 1]);

// Apply scale transformation
var scaleVec = update_scale();
mat4.scale(mvMatrix, scaleVec);

// Apply shear transformation
var shearVals = update_shear();
applyShear(mvMatrix, shearVals[0], shearVals[1], shearVals[2]);

setMatrixUniforms();

// Setup teapot position data
gl.bindBuffer(gl.ARRAY_BUFFER, teapotVertexPositionBuffer);
gl.vertexAttribPointer(shaderProgram.vertexPositionAttribute,
                       teapotVertexPositionBuffer.itemSize,
                       gl.FLOAT,
                       false,
                       0,
                       0);

// Setup teapot front color data
gl.bindBuffer(gl.ARRAY_BUFFER, teapotVertexFrontColorBuffer);
gl.vertexAttribPointer(shaderProgram.vertexFrontColorAttribute,
                       teapotVertexFrontColorBuffer.itemSize,
                       gl.FLOAT,
                       false,
                       0,
                       0);

// Setup teapot normal data
gl.bindBuffer(gl.ARRAY_BUFFER, teapotVertexNormalBuffer);
```

```
  gl.vertexAttribPointer(shaderProgram.vertexNormalAttribute,
                         teapotVertexNormalBuffer.itemSize,
                         gl.FLOAT,
                         false,
                         0,
                         0);

  // Setup ambient light and light position
  gl.uniform1f(gl.getUniformLocation(shaderProgram, "Ka"), ka);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightLoc1"),
  light_locations1);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightLoc2"),
  light_locations2);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightColor1"),
  light_colors1);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightColor2"),
  light_colors2);

  gl.drawArrays(gl.TRIANGLES, 0, teapotVertexPositionBuffer.numItems);

  // 恢復矩陣狀態
  mat4.set(mvMatrixCopy, mvMatrix);
```

}

// 繪製袋鼠的函數
function drawKangaroo(posX, posY, posZ) {
// 保存當前矩陣狀態
var mvMatrixCopy = mat4.create();
mat4.set(mvMatrix, mvMatrixCopy);

```
  // Setup Model-View Matrix
  mat4.identity(mvMatrix);
  mat4.translate(mvMatrix, [posX, posY, posZ]);

  // Apply translation transformation
  var transVec = update_trans();
  mat4.translate(mvMatrix, transVec);

  var rotateVec_init = [0,0,0];
```

```
mat4.rotate(mvMatrix, degToRad(rotateVec_init[0]), [1, 0, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec_init[1]), [0, 1, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec_init[2]), [0, 0, 1]);

// Apply rotation transformation
var rotateVec = update_rotate();
mat4.rotate(mvMatrix, degToRad(rotateVec[0]), [1, 0, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec[1] + teapotAngle), [0, 1, 0]);
mat4.rotate(mvMatrix, degToRad(rotateVec[2]), [0, 0, 1]);

// Apply scale transformation
var scaleVec = update_scale();
mat4.scale(mvMatrix, scaleVec);

// Apply shear transformation
var shearVals = update_shear();
applyShear(mvMatrix, shearVals[0], shearVals[1], shearVals[2]);

setMatrixUniforms();

// Setup kangaroo position data
gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexPositionBuffer);
gl.vertexAttribPointer(shaderProgram.vertexPositionAttribute,
                       kangarooVertexPositionBuffer.itemSize,
                       gl.FLOAT,
                       false,
                       0,
                       0);

// Setup kangaroo front color data
gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexFrontColorBuffer);
gl.vertexAttribPointer(shaderProgram.vertexFrontColorAttribute,
                       kangarooVertexFrontColorBuffer.itemSize,
                       gl.FLOAT,
                       false,
                       0,
                       0);

// Setup kangaroo normal data
gl.bindBuffer(gl.ARRAY_BUFFER, kangarooVertexNormalBuffer);
```

```
  gl.vertexAttribPointer(shaderProgram.vertexNormalAttribute,
                         kangarooVertexNormalBuffer.itemSize,
                         gl.FLOAT,
                         false,
                         0,
                         0);

  // Setup ambient light and light position (相同的光照設置)
  gl.uniform1f(gl.getUniformLocation(shaderProgram, "Ka"), ka);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightLoc1"),
  light_locations1);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightLoc2"),
  light_locations2);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightColor1"),
  light_colors1);
  gl.uniform3fv(gl.getUniformLocation(shaderProgram, "lightColor2"),
  light_colors2);

  gl.drawArrays(gl.TRIANGLES, 0, kangarooVertexPositionBuffer.numItems);

  // 恢復矩陣狀態
  mat4.set(mvMatrixCopy, mvMatrix);
```

}

這些修改實現了以下效果：

1. 加載和處理袋鼠模型，與處理茶壺模型的方式相同
2. 將畫面分為左右兩部分，左側顯示茶壺，右側顯示袋鼠
3. 所有控制參數（著色模式、光照、變換、剪裁平面）將同時應用於兩個模型
4. 動畫效果將同樣作用於兩個模型

袋鼠模型在c:\Tommy\WebGL\ICG_HW1\model\Kangaroo.json